Inspiring Excellence

CSE422: Artificial Intellegence

Lab Project Report

Project Title: Academic Success Prediction

Name	ID
Nafisa Hasan	22299141
Swapnil Rob	22299138

Table of Contents

Introduction

The main goal of this project is to predict a student's academic outcome using different academic, demographic, and socio-economic features from the dataset. The output "Target" has three possible outcomes: Dropout, Enrolled, and Graduate.

This type of prediction can be useful for universities because it can help identify students who may need support early (for example financial aid, counselling, academic help, etc.). In this project, we trained multiple machine learning models and compared their performance. We also treated the same problem as an unsupervised task using clustering to see if natural groups exist in the data.

Dataset Description

Dataset overview:

The dataset used in this project is academic_success_dataset.csv. Initially, the dataset contained 4424 rows and 27 columns. However, two columns were unnamed and did not contain meaningful information, so they were excluded from analysis. Additionally, rows with missing target values were removed for supervised learning, resulting in 3971 valid data points.

There are 24 input features and one output feature named Target. The target variable has three distinct categories and an additional : Dropout, Enrolled, and Graduate. During preprocessing, missing values were filled before separating the target variable, which resulted in missing target values being encoded as a valid class.As a result, the classification task was treated as a four-class classification problem, where the additional class corresponds to records that originally had missing target values.Since the output consists of multiple discrete classes rather than continuous values, this problem is a multiclass classification problem.

Feature types and Encoding :

The dataset contains both numerical and categorical features. Numerical features represent academic and institutional measurements, while categorical features represent different coded categories. Categorical features must be encoded into numerical values because machine learning models cannot work directly with non-numeric data.

Machine learning models cannot process categorical text values directly, so these features must be converted into numerical form.

```python
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
import warnings as wr # for Handle Error
le=LabelEncoder()
wr.filterwarnings('ignore')
for col in df.columns:
    if df[col].dtype==np.number:
        continue
    df[col] = df[col].astype(str)
    df[col]=le.fit_transform(df[col])
df
```

This loop ensures that all categorical columns are converted into numeric labels. This step is necessary for consistent training across all machine learning models.

Correlation Analysis :

Correlation analysis was performed to understand the relationship between different features and the target variable. After encoding all categorical variables, a correlation matrix was generated and visualized using a heatmap. This helps identify which features are positively or negatively associated with academic success.

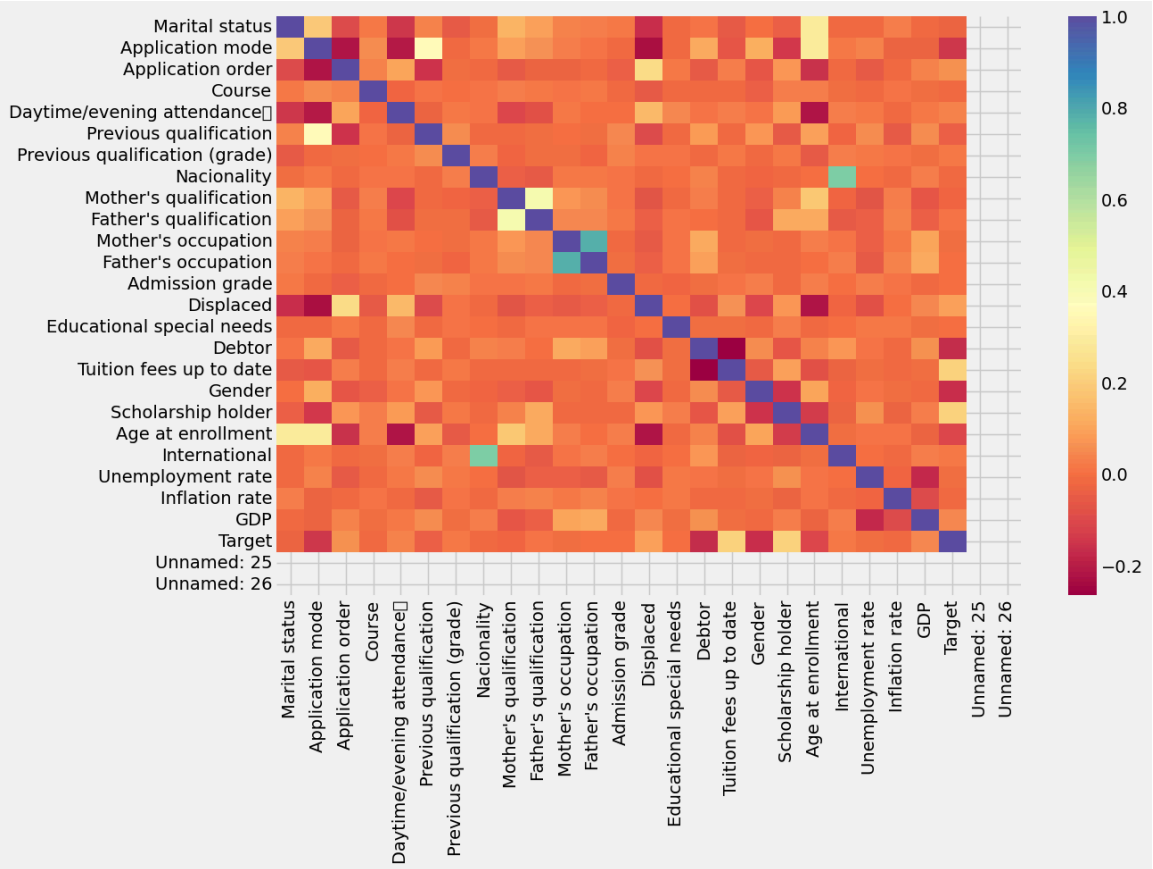


Figure 1: Correlation heatmap of numerical features

From the heatmap, it can be observed that features related to tuition fee status, scholarship holding, and debtor status show stronger correlations with the target variable. This indicates that financial stability plays a significant role in determining whether a student graduates or drops out. Academic-related attributes also show moderate influence on the outcome.

Class Imbalance:

The distribution of the target variable was analyzed to check for class imbalance. After cleaning the dataset, the number of samples in each class was not equal.

- Graduate: 1979
- Dropout: 1273
- Enrolled: 719
- Missing: 453

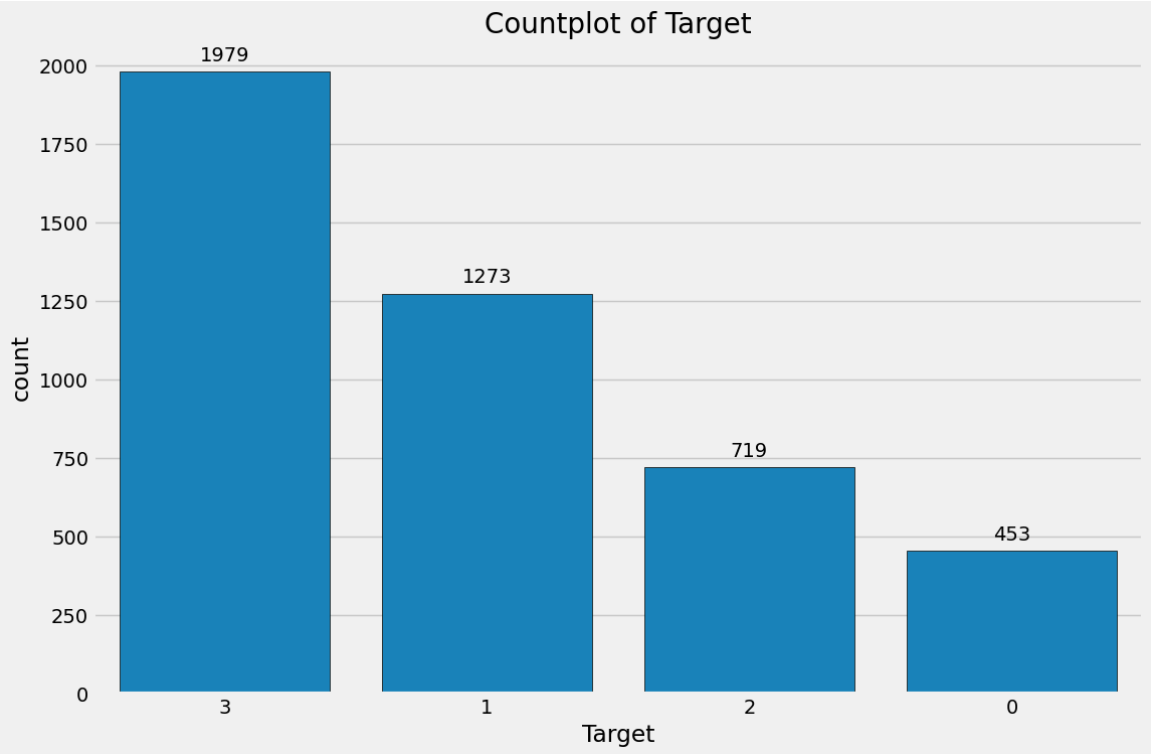


Figure 2: Bar chart of target class distribution

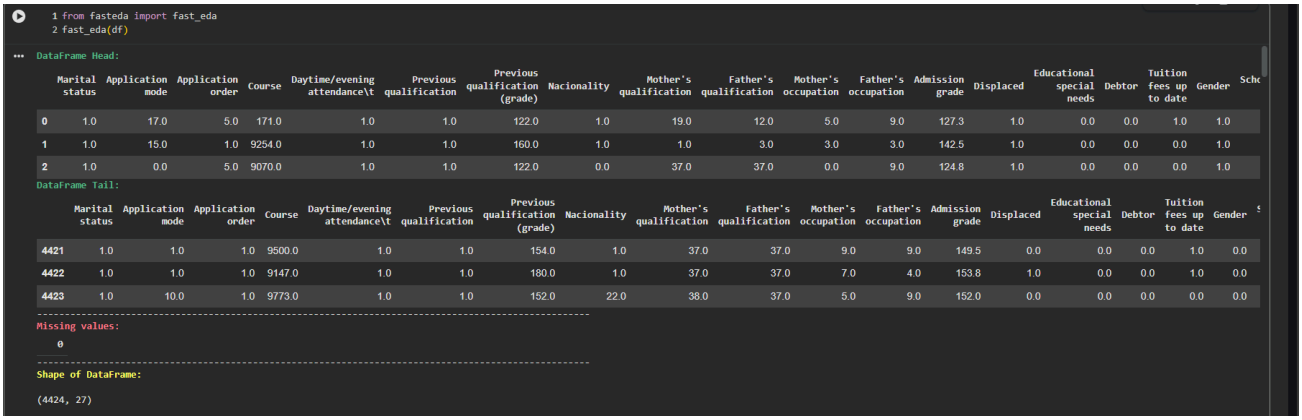
This shows that the dataset is imbalanced, especially for the Enrolled class. Class imbalance is important because models may favor the majority class and still achieve high accuracy while performing poorly on minority classes.

Exploratory Data Analysis (EDA):

Exploratory Data Analysis was conducted to gain a better understanding of the dataset structure, missing values, and overall feature behavior. The FastEDA library was used to generate a summarized view of the dataset efficiently.

```
1 from fasteda import fast_eda
2 fast_eda(df)
```

The EDA results show the presence of missing values in several features and highlight the need for data cleaning before model training.



```
1 from fasteda import fast_eda
2 fast_eda(df)
```

DataFrame Head:

	Marital status	Application mode	Application order	Course	Daytime/evening attendance\t	Previous qualification	Previous qualification (grade)	Nacionality	Mother's qualification	Father's qualification	Mother's occupation	Father's occupation	Admission grade	Displaced	Educational special needs	Debtor	Tuition fees up to date	Gender	Schc
0	1.0	17.0	5.0	171.0	1.0	1.0	122.0	1.0	19.0	12.0	5.0	9.0	127.3	1.0	0.0	0.0	1.0	1.0	
1	1.0	15.0	1.0	9254.0	1.0	1.0	160.0	1.0	1.0	3.0	3.0	3.0	142.5	1.0	0.0	0.0	0.0	1.0	
2	1.0	0.0	5.0	9070.0	1.0	1.0	122.0	0.0	37.0	37.0	0.0	9.0	124.8	1.0	0.0	0.0	0.0	1.0	

DataFrame Tail:

	Marital status	Application mode	Application order	Course	Daytime/evening attendance\t	Previous qualification	Previous qualification (grade)	Nacionality	Mother's qualification	Father's qualification	Mother's occupation	Father's occupation	Admission grade	Displaced	Educational special needs	Debtor	Tuition fees up to date	Gender	S
4421	1.0	1.0	1.0	9500.0	1.0	1.0	154.0	1.0	37.0	37.0	9.0	9.0	149.5	0.0	0.0	0.0	1.0	0.0	
4422	1.0	1.0	1.0	9147.0	1.0	1.0	180.0	1.0	37.0	37.0	7.0	4.0	153.8	1.0	0.0	0.0	1.0	0.0	
4423	1.0	10.0	1.0	9773.0	1.0	1.0	152.0	22.0	38.0	37.0	5.0	9.0	152.0	0.0	0.0	0.0	0.0	0.0	

Missing values:

0

Shape of DataFrame:

(4424, 27)

Figure 3: EDA summary / missing value visualization

Dataset Pre-processing

Handling Missing Values:

Missing values can negatively affect model training and may cause biased predictions. Therefore, handling missing data is a critical preprocessing step.

```
1 df.fillna(0,inplace=True)
2
```

This approach replaces missing values with zero to ensure that models can train without errors. Rows with missing target values were excluded beforehand to maintain supervised learning validity.

```
1 df.isnull().sum()
```

	0
Marital status	423
Application mode	483
Application order	426
Course	465
Daytime/evening attendance\t	440
Previous qualification	434
Previous qualification (grade)	472
Nacionality	446

Mother's qualification	414
Father's qualification	450
Mother's occupation	436
Father's occupation	425
Admission grade	462
Displaced	442
Educational special needs	448
Debtor	452
Tuition fees up to date	426
Gender	437
Scholarship holder	450
Age at enrollment	444
International	428
Unemployment rate	429
Inflation rate	422
GDP	456
Target	453
Unnamed: 25	4424
Unnamed: 26	4424

dtype: int64

Figure 4: Missing value count before preprocessing

Feature Scaling:

Some machine learning models, particularly Logistic Regression and Neural Networks, are sensitive to feature scales. Features with larger ranges can dominate the learning process if scaling is not applied.

```
1 from sklearn.preprocessing import LabelEncoder, OneHotEncoder
```

Scaling improves model stability and convergence and helps ensure fair contribution from all features.

Dataset Splitting

The cleaned dataset was split into training and testing sets using a 70:30 ratio. Stratified splitting was applied to preserve the class distribution in both sets, which is important due to class imbalance.

- Training set: 2779 samples
- Testing set: 1179 samples

```
1 xtrain,xtest,ytrain,ytest=train_test_split(X,Y,test_size=0.3,random_state=42)
```

Model Training & Testing

In this section, multiple machine learning models were trained and tested to evaluate their performance in predicting academic outcomes. Three supervised learning models were used: K-Nearest Neighbors (KNN), Logistic Regression, and a Neural Network. In addition, the dataset was also analyzed using unsupervised learning through K-Means clustering. All supervised models were trained using the training dataset and evaluated on the test dataset.

K-Nearest Neighbors (KNN):

K-Nearest Neighbors (KNN) is a supervised learning algorithm that classifies a data point based on the majority class among its nearest neighbors. It is a simple yet effective model that relies on distance calculations in feature space. In this project, KNN was used to classify students into Dropout, Enrolled, and Graduate categories.

The model was trained using k = 5 neighbors, which was chosen as a reasonable balance between bias and variance. The training was performed using the training dataset (xtrain, ytrain), and predictions were made on the test dataset (xtest).

```python
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(xtrain, ytrain)

ypred_knn = knn.predict(xtest)
yprob_knn = knn.predict_proba(xtest)

```

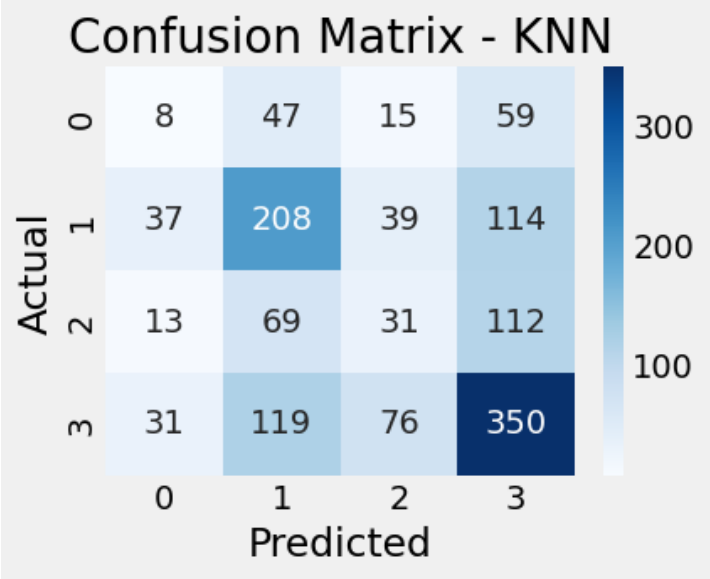


Figure 5: KNN confusion matrix (from confusion_matrix)

Logistic Regression:

Logistic Regression is a supervised classification algorithm that models the relationship between input features and the probability of class membership. Although it is a linear model, it often performs well as a baseline classifier and is easy to interpret. In this project, Logistic Regression was applied to the same training and testing datasets as KNN.

```
from sklearn.linear_model import LogisticRegression

log = LogisticRegression(max_iter=3000)
log.fit(xtrain, ytrain)

ypred_log = log.predict(xtest)
yprob_log = log.predict_proba(xtest)

```

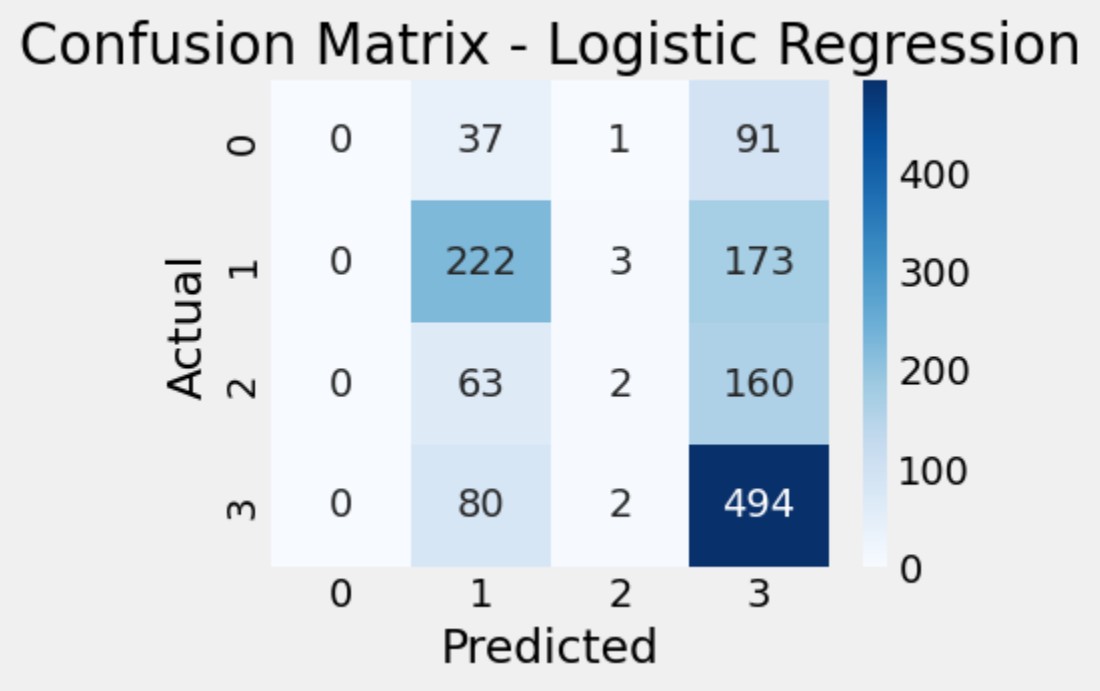


Figure 6: Logistic Regression confusion matrix

Neural Network:

A Neural Network model was used to capture more complex and non-linear relationships between the input features and the target variable. Neural Networks are particularly useful when interactions between features are not

purely linear. In this project, a Multi-Layer Perceptron (MLP)–style Neural Network was trained using the same dataset split.

The Neural Network achieved performance comparable to Logistic Regression, suggesting that while non-linear patterns exist in the data, they may not be extremely strong due to dataset size, noise, or class imbalance.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical


tf.random.set_seed(42)
np.random.seed(42)
num_classes = len(np.unique(ytrain))




if num_classes > 2:
    ytrain_fit = to_categorical(ytrain, num_classes=num_classes)
    ytest_fit = to_categorical(ytest, num_classes=num_classes)
else:
    ytrain_fit = ytrain
    ytest_fit = ytest
```

```
nn = Sequential()

nn.add(Dense(30, activation="relu", input_shape=(xtrain.shape[1],)))

nn.add(Dense(50, activation="relu"))



if num_classes == 2:

    nn.add(Dense(1, activation="sigmoid"))

    nn.compile(optimizer="adam", loss="binary_crossentropy",
metrics=["accuracy"])

else:

    nn.add(Dense(num_classes, activation="softmax"))

    nn.compile(optimizer="adam", loss="categorical_crossentropy",
metrics=["accuracy"])



history = nn.fit(

    xtrain, ytrain_fit,

    epochs=30,

    batch_size=32,

    validation_split=0.2,

    verbose=1

)




pred_nn = nn.predict(xtest)



if num_classes == 2:

    yprob_nn = pred_nn.flatten()
```

```
    ypred_nn = (yprob_nn >= 0.5).astype(int)
else:
    yprob_nn = pred_nn
    ypred_nn = np.argmax(pred_nn, axis=1)
```

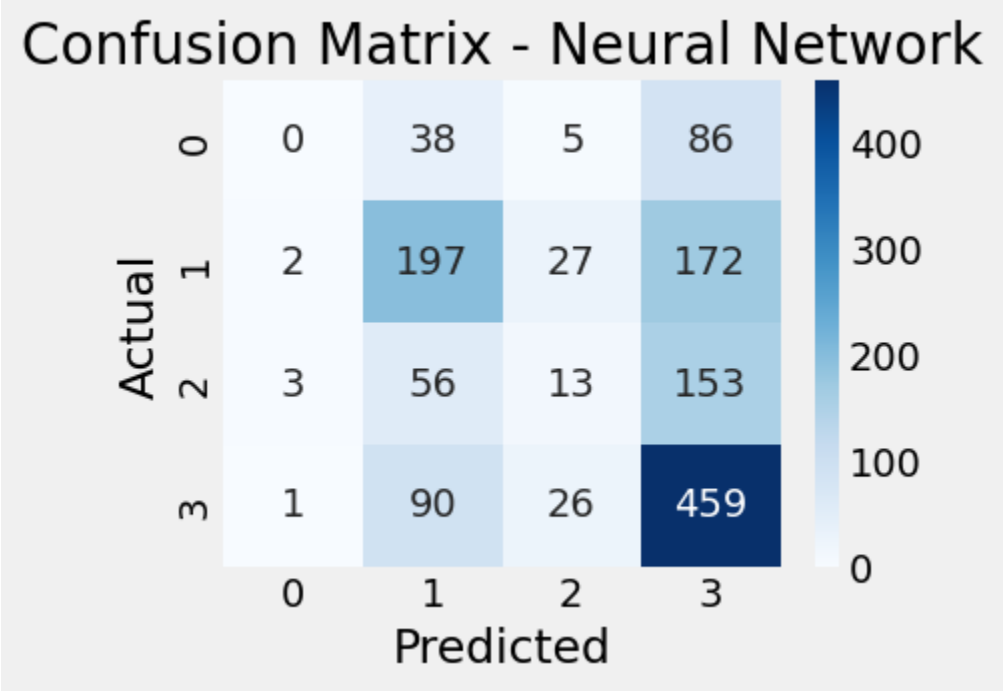


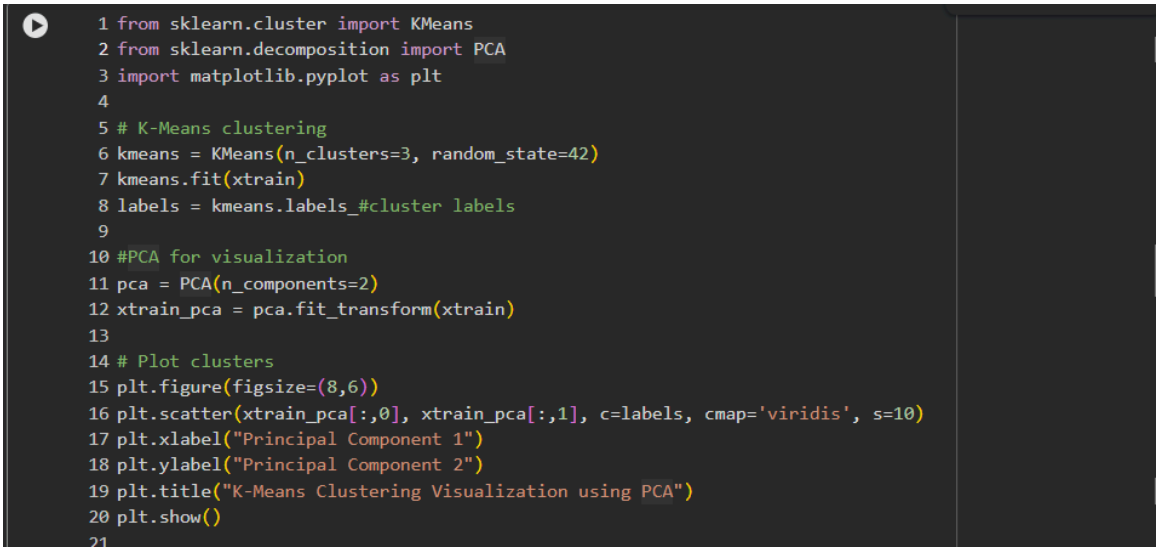
Figure 7: Neural Network confusion matrix

K-Means Clustering (Unsupervised Learning):

After completing supervised learning, K-Means clustering was applied as an unsupervised learning technique. Unlike supervised models, K-Means does not use the target labels and instead groups data points based on similarity in the feature space. The goal was to examine whether students form natural

clusters based on their attributes.

The number of clusters was set to k = 3, which aligns with the three academic outcome categories. Although the clustering does not use the target variable, this choice allows an intuitive comparison between clusters and actual outcomes.



```python
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# K-Means clustering
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(xtrain)
labels = kmeans.labels_#cluster labels

#PCA for visualization
pca = PCA(n_components=2)
xtrain_pca = pca.fit_transform(xtrain)

# Plot clusters
plt.figure(figsize=(8,6))
plt.scatter(xtrain_pca[:,0], xtrain_pca[:,1], c=labels, cmap='viridis', s=10)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("K-Means Clustering Visualization using PCA")
plt.show()
```

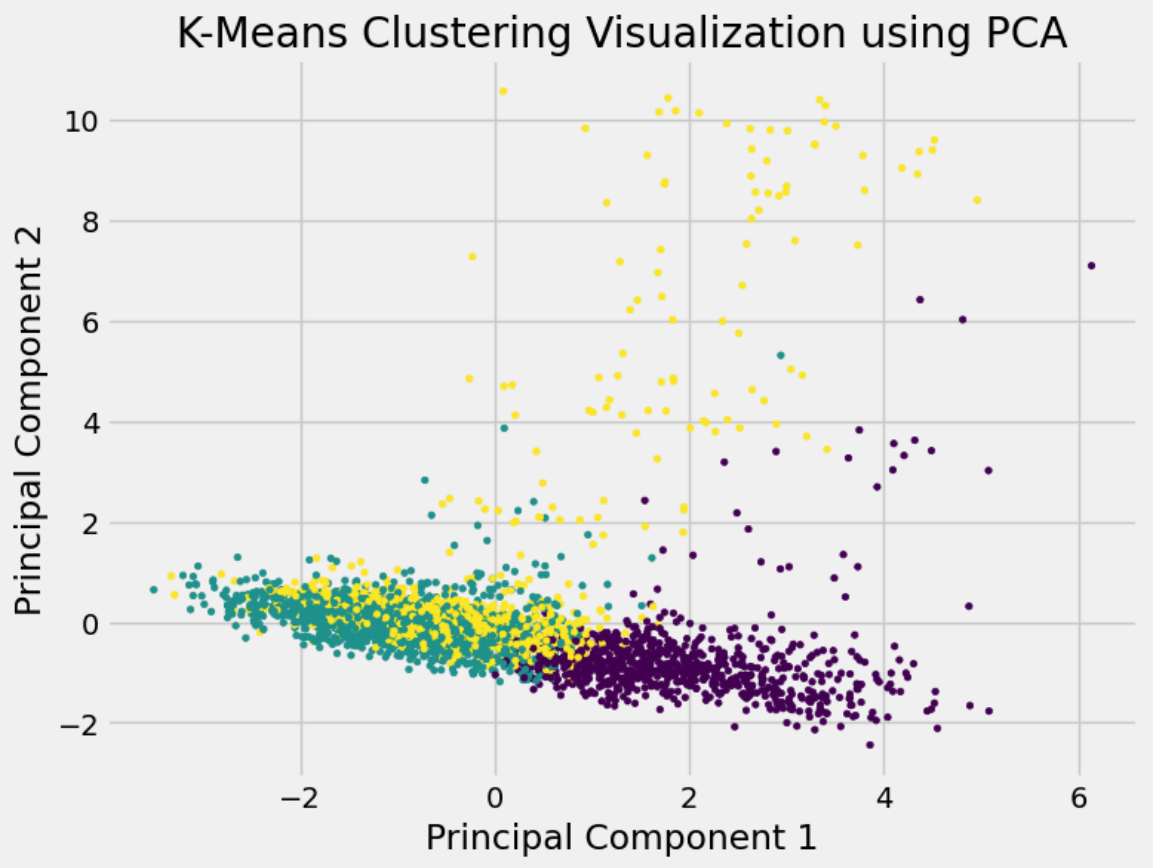


Figure 8: K-Means clusters

Model Selection and Comparison Analysis:

To compare the supervised learning models, multiple evaluation metrics were analyzed, including accuracy, precision, recall, confusion matrices, and ROC-AUC scores. Since the dataset is imbalanced, accuracy alone was not sufficient to judge model performance.

From the comparison, Logistic Regression achieved the best overall performance, followed closely by the Neural Network. KNN performed comparatively worse, likely due to its sensitivity to feature space and class imbalance. Confusion matrices showed that Logistic Regression and the Neural Network handled minority classes more effectively.

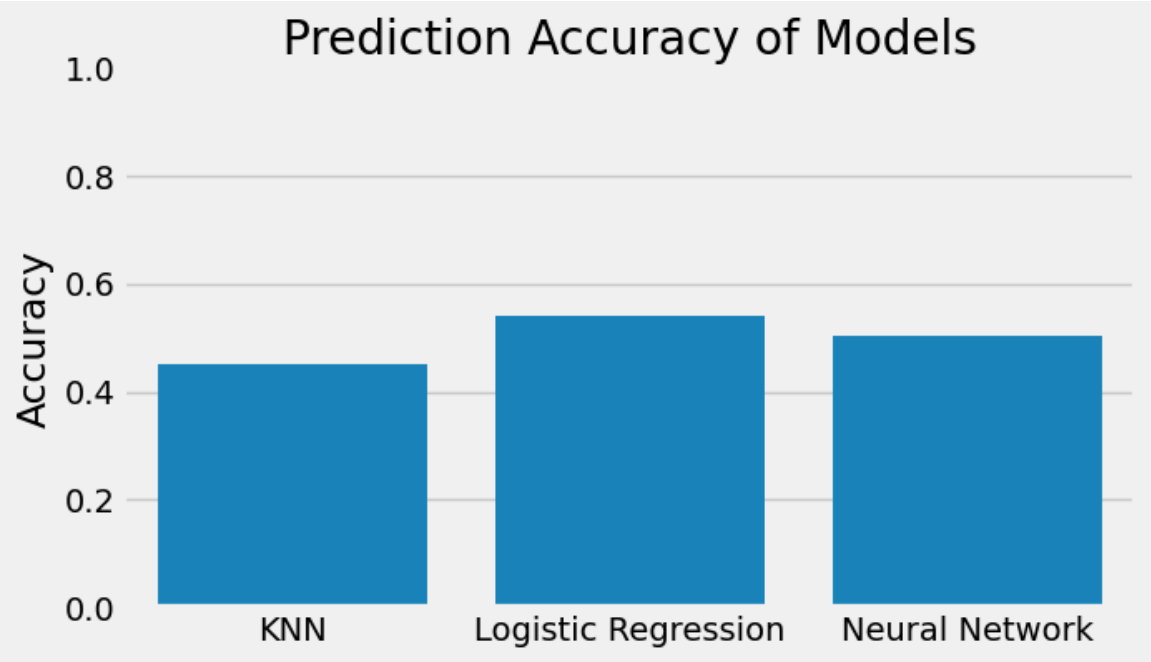
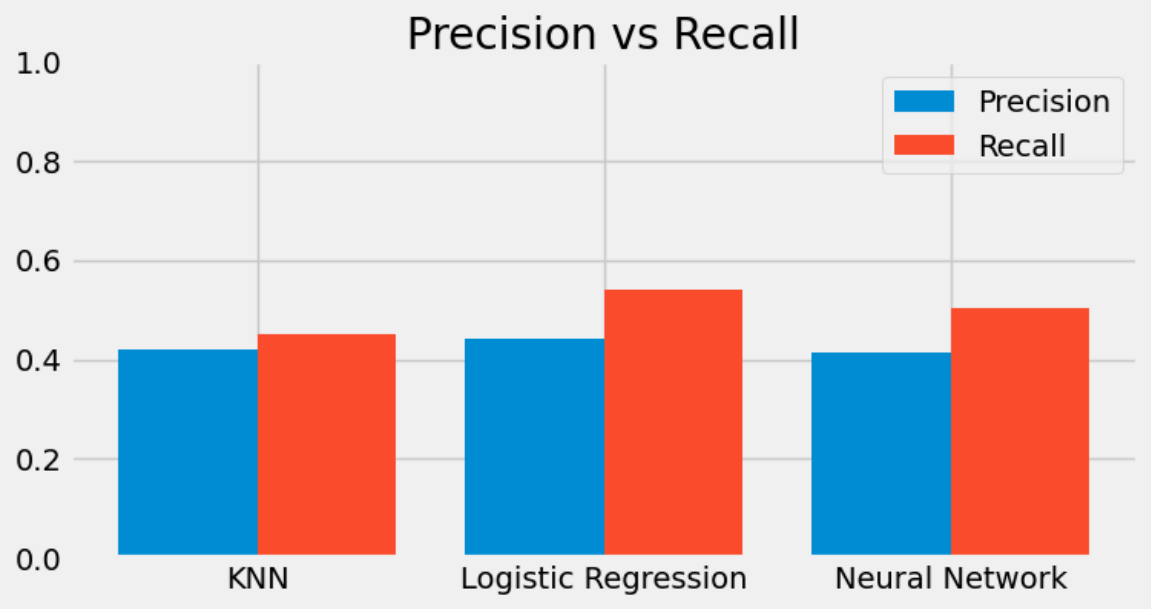
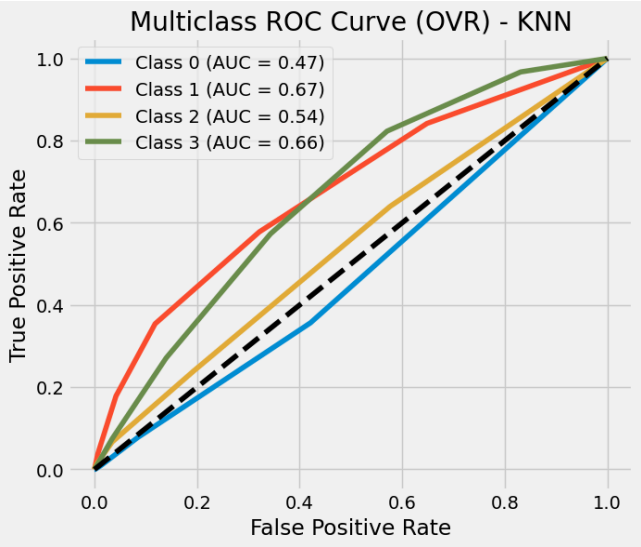


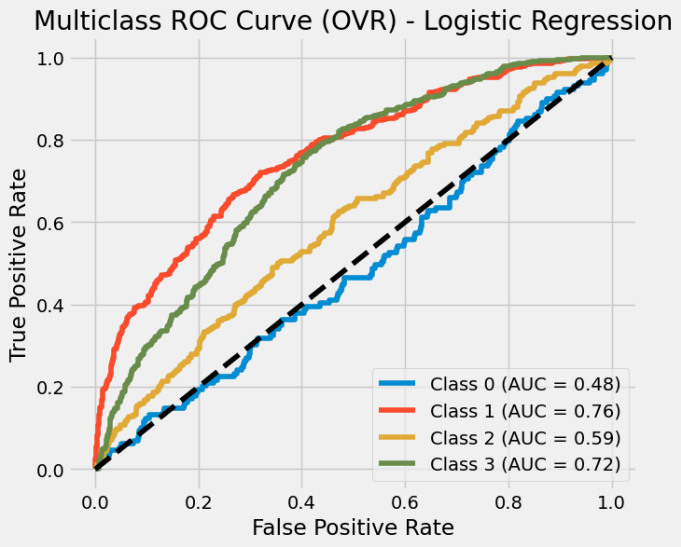
Figure 9: Accuracy comparison bar chart




```
KNN Precision = 0.4211371489258355 | Recall = 0.4495481927710843
Logistic Regression Precision = 0.4412664375676543 | Recall = 0.5406626506024096
Neural Network Precision = 0.41481691605250337 | Recall = 0.5037650602409639
```

Figure 10: Precision vs Recall





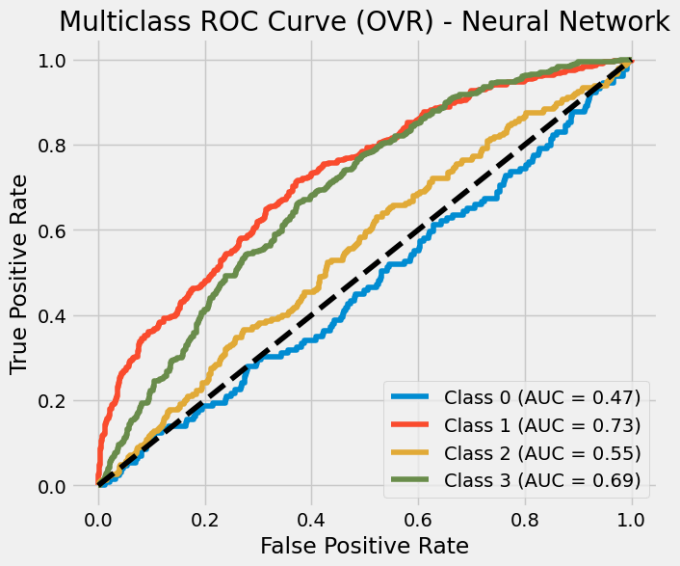


Figure 11: ROC curve for all supervised model

KNN Macro AUC (OVR) = 0.5870688816182195

Logistic Regression Macro AUC (OVR) = 0.6410610405493239

Neural Network Macro AUC (OVR) = 0.6085248416015436

Multiclass ROC: macro AUC printed (OVR).

Conclusion

In this project, machine learning techniques were applied to predict students' academic outcomes. Three supervised learning models—KNN, Logistic Regression, and a Neural Network were trained and evaluated, along with an unsupervised K-Means clustering approach. Among the supervised models, Logistic Regression performed the best overall, achieving the highest accuracy and ROC-AUC score.

The Neural Network also demonstrated strong performance, indicating the presence of non-linear patterns in the data. However, the simpler Logistic Regression model proved more stable and effective for this dataset.
K-Means clustering provided additional insight into the structure of the data, although the clusters did not perfectly align with actual academic outcomes.

The main challenges faced in this project included handling class imbalance, selecting appropriate evaluation metrics, and interpreting unsupervised clustering results. Overall, the project highlights the importance of proper preprocessing, careful model selection, and thorough evaluation when applying machine learning to real-world educational data.